FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Computer System and Architecture

Machine Problem

FLAB3

Kurt Lester Carolino	Doc. Hadji Tejuco
Name of Student	Name of Professor
October 29, 2026	October 29, 2026
Data Performed	Date Submitted

MIDTERM EXAM 1

FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

List the steps that you perform to come up with the program.

Analyzed the COR layout: Divided the form into three parts: the top part for student info (user input), the middle part for subjects and schedules (fixed text), and the bottom part for the assessment fees (user input and math calculations).

Set up the .data and .bss sections: Put all the text prompts, labels, the fixed schedule table, and the constant tuition fee (44,268.00) inside .data, and reserved empty memory variables for the user's answers and math results inside .bss.

Created Macros for repeated tasks: Wrote macros (PRINT_ZSTR, PROMPT_TEXT, PROMPT_FEE, and PRINT_FEE_ROW) to handle printing text and reading keyboard input so I did not have to rewrite Linux system calls (int 0x80) every time.

Handled decimal math using centavos: Since basic assembly registers work with whole integers, I multiplied all money values by 100 (converting pesos to centavos) so the program could accurately calculate .00, .40, and .30 decimals without floating-point errors.

Wrote Procedures for each major step: Used call and ret procedures to organize the program into clear steps: getting inputs (get_user_inputs), computing the total and percentages (calculate_assessment), and printing the three sections of the COR.

Tested and formatted the output: Assembled and linked the program in the Linux terminal using nasm and ld, then adjusted the spacing so the two columns at the top and the fee table at the bottom lined up properly.

Paste the source code and explain the functionalities and take a screenshot of the final output.

data and .bss Sections: The .data section stores fixed strings like questions, the ASCII schedule table, and the fixed tuition fee (4426800 centavos). The .bss section reserves buffers for the student's typed text and variables for the calculated fees.

Macros (%macro):

PRINT_ZSTR: Passes a string to the printing procedure to display text on the screen.

PROMPT_TEXT: Prints a question, reads what the user types using sys_read (eax = 3), and removes the extra Enter key (newline) at the end.

PROMPT_FEE: Reads a fee amount from the user, converts the text into a number, and saves it in a variable.

PRINT_FEE_ROW: Prints a fee label alongside its formatted money value.

Main Procedures (call / ret):

get_user_inputs: Asks the user to type their personal details (Status, Student #, Name, Phone, Zip Code, College, Program, Year, Term, Address) and the variable fees (Miscellaneous, ID Validation, and Laboratory fees).

calculate_assessment: Adds the fixed Tuition Fee (44,268.00) and all the user-entered fees to get the total_assessment. It then uses multiplication (mul) and division (div) to compute the Down Payment (Total × 40 ÷ 100), Midterm (Total × 30 ÷ 100), and Final payment (Total × 30 ÷ 100).

print_top_section, print_schedule_section, and print_assessment_section: Prints the completed registration form in order—aligning the student details into left and right columns, displaying the static schedule table, and printing the final assessment breakdown.

Helper Procedures:

parse_currency: Converts the typed fee string (like "12305.00") into an integer in centavos (1230500) so the CPU can add and multiply it.

print_currency: Converts the calculated centavo numbers back into readable currency format by inserting the decimal point (.) and commas (,) before printing.

pad_to_col_56: Counts how many characters were printed on the left side of the student info header and prints spaces so the right column lines up evenly.

Program Description

This program is a 32-bit x86 NASM Assembly application that recreates a student's Certificate of Registration (COR) and calculates their school fees. It uses macros and procedures to ask the user for their student information and lab/miscellaneous fees, prints a fixed table of their subjects and class schedule, and calculates the Total Assessment along with the 40% Down Payment, 30% Midterm, and 30% Final payment installments.

PROGRAM OUTPUT:

MIDTERM EXAM 2

FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

```
=== ENTER STUDENT INFORMATION ===
Enrollment Status : ENROLLED
Student #         : 202510161
Name              : Kurt Lester A. Carolino
Phone             : 09913575840
Zip Code          : 1485
Classification    : N/A
College           : COMPUTER STUDIES
Program           : BSCSSE
Year Level        : 2
Term & SY         : 1st term, SY 2026-2027
Address           : 335 TAGANAHAN STREET, BAGUMBAYAN SOUTH

=== ENTER ASSESSMENT FEES (Tuition fixed at 44,268.00) ===
Miscellaneous Fee                  : 12,306.00
ID Validation Fee                  : 80.00
ITE Computer Lab Fee (CCS0021)     : 4,008.00
ITE Computer Lab Fee (CS0003)      : 4,008.00
ITE Computer Lab Fee (CS0070)      : 4,008.00
Science Lab Fee (GED0081)          : 4,104.00

Enrollment Status : ENROLLED
Student # : 202510161                          College : COMPUTER STUDIES   Program : BSCSSE
Name : Kurt Lester A. Carolino                 Year Level : 2   1st term, SY 2026-2027
Phone : 09913575840   Zip Code : 1485          Address : 335 TAGANAHAN STREET, BAGUMBAYAN SOUTH
Classification : N/A

+-----------------------------------------------------------------------------------------------------------------------------------------------+
| Courses   Title                                Section   Units   Days    Time                                          Room                 |
+-----------------------------------------------------------------------------------------------------------------------------------------------+
| CCS0021   INFORMATION MANAGEMENT (LEC)         TN24        2     T       16:00:00-18:40:00                             ONLINE               |
| CCS0021L  INFORMATION MANAGEMENT (LAB)         TN24        1     F       16:00:00-18:50:00                             ONLINE               |
| CS0001    DISCRETE STRUCTURES 1                TN24        3     S / W   15:00:00-16:50:00 / 15:00:00-16:50:00   F710 / F710          |
```

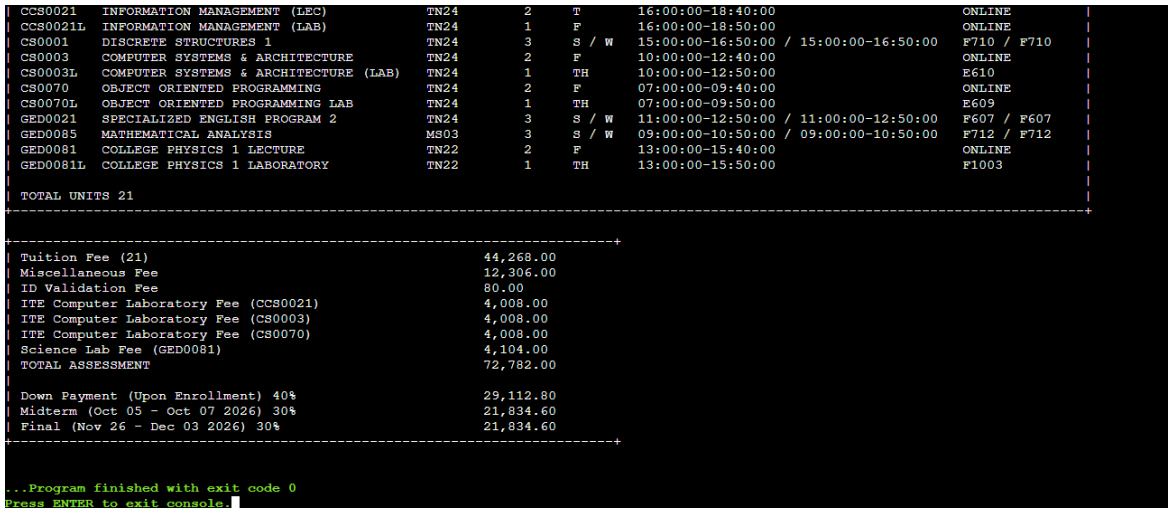
MIDTERM EXAM 3

FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Enrollment Status : ENROLLED
Student # : 202510161 College : COMPUTER STUDIES Program : BSCSSE
Name : Kurt Lester A. Carolino Year Level : 2 1st term, SY 2026-2027
Phone : 09913575840 Zip Code : 1485 Address : 335 TAGANAHAN STREET, BAGUMBAYAN SOUTH
Classification : N/A

Courses	Title	Section	Units	Days	Time	Room
CCS0021	INFORMATION MANAGEMENT (LEC)	TN24	2	T	16:00:00-18:40:00	ONLINE
CCS0021L	INFORMATION MANAGEMENT (LAB)	TN24	1	F	16:00:00-18:50:00	ONLINE
CS0001	DISCRETE STRUCTURES 1	TN24	3	S / W	15:00:00-16:50:00 / 15:00:00-16:50:00	F710 / F710
CS0003	COMPUTER SYSTEMS & ARCHITECTURE	TN24	2	F	10:00:00-12:40:00	ONLINE
CS0003L	COMPUTER SYSTEMS & ARCHITECTURE (LAB)	TN24	1	TH	10:00:00-12:50:00	E610
CS0070	OBJECT ORIENTED PROGRAMMING	TN24	2	F	07:00:00-09:40:00	ONLINE
CS0070L	OBJECT ORIENTED PROGRAMMING LAB	TN24	1	TH	07:00:00-09:50:00	E609
GED0021	SPECIALIZED ENGLISH PROGRAM 2	TN24	3	S / W	11:00:00-12:50:00 / 11:00:00-12:50:00	F607 / F607
GED0085	MATHEMATICAL ANALYSIS	MS03	3	S / W	09:00:00-10:50:00 / 09:00:00-10:50:00	F712 / F712
GED0081	COLLEGE PHYSICS 1 LECTURE	TN22	2	F	13:00:00-15:40:00	ONLINE
GED0081L	COLLEGE PHYSICS 1 LABORATORY	TN22	1	TH	13:00:00-15:50:00	F1003
TOTAL UNITS 21						

Tuition Fee (21)	44,268.00
Miscellaneous Fee	12,306.00
ID Validation Fee	80.00
ITE Computer Laboratory Fee (CCS0021)	4,008.00
ITE Computer Laboratory Fee (CS0003)	4,008.00
ITE Computer Laboratory Fee (CS0070)	4,008.00
Science Lab Fee (GED0081)	4,104.00



```
| CCS0021   INFORMATION MANAGEMENT (LEC)           TN24        2     T       16:00:00-18:40:00                       ONLINE          |
| CCS0021L  INFORMATION MANAGEMENT (LAB)           TN24        1     F       16:00:00-18:50:00                       ONLINE          |
| CS0001    DISCRETE STRUCTURES 1                  TN24        3     S / W   15:00:00-16:50:00 / 15:00:00-16:50:00   F710 / F710    |
| CS0003    COMPUTER SYSTEMS & ARCHITECTURE        TN24        2     F       10:00:00-12:40:00                       ONLINE          |
| CS0003L   COMPUTER SYSTEMS & ARCHITECTURE (LAB)  TN24        1     TH      10:00:00-12:50:00                       E610            |
| CS0070    OBJECT ORIENTED PROGRAMMING            TN24        2     F       07:00:00-09:40:00                       ONLINE          |
| CS0070L   OBJECT ORIENTED PROGRAMMING LAB        TN24        1     TH      07:00:00-09:50:00                       E609            |
| GED0021   SPECIALIZED ENGLISH PROGRAM 2          TN24        3     S / W   11:00:00-12:50:00 / 11:00:00-12:50:00   F607 / F607    |
| GED0085   MATHEMATICAL ANALYSIS                  MS03        3     S / W   09:00:00-10:50:00 / 09:00:00-10:50:00   F712 / F712    |
| GED0081   COLLEGE PHYSICS 1 LECTURE              TN22        2     F       13:00:00-15:40:00                       ONLINE          |
| GED0081L  COLLEGE PHYSICS 1 LABORATORY           TN22        1     TH      13:00:00-15:50:00                       F1003           |
|                                                                                                                                  |
| TOTAL UNITS 21                                                                                                                   |
+----------------------------------------------------------------------------------------------------------------------------------+

+-------------------------------------------------------------------+
| Tuition Fee (21)                                    44,268.00
| Miscellaneous Fee                                   12,306.00
| ID Validation Fee                                   80.00
| ITE Computer Laboratory Fee (CCS0021)               4,008.00
| ITE Computer Laboratory Fee (CS0003)                4,008.00
| ITE Computer Laboratory Fee (CS0070)                4,008.00
| Science Lab Fee (GED0081)                           4,104.00
| TOTAL ASSESSMENT                                    72,782.00
|
| Down Payment (Upon Enrollment) 40%                  29,112.80
| Midterm (Oct 05 - Oct 07 2026) 30%                  21,834.60
| Final (Nov 26 - Dec 03 2026) 30%                    21,834.60
+-------------------------------------------------------------------+


...Program finished with exit code 0
Press ENTER to exit console.
```

SOURCE CODE:

```
; The .data section is where we store fixed text (like questions and labels)
; and numbers that do not change.
; Note: "10" means a new line (Enter key), and "0" marks the end of the text.
section .data
    ; --- Questions for Student Details (Top Part) ---
    hdr_input_top       db "=== ENTER STUDENT INFORMATION ===", 10, 0
    p_status            db "Enrollment Status : ", 0
    p_student_no        db "Student #         : ", 0
    p_name              db "Name              : ", 0
    p_phone             db "Phone             : ", 0
    p_zip               db "Zip Code          : ", 0
    p_class             db "Classification    : ", 0
    p_college           db "College           : ", 0
    p_program           db "Program           : ", 0
    p_year              db "Year Level        : ", 0
    p_term_sy           db "Term & SY         : ", 0
    p_address           db "Address           : ", 0

    ; --- Questions for School Fees (Bottom Part) ---
    hdr_input_fees      db 10, "=== ENTER ASSESSMENT FEES (Tuition fixed at
44,268.00) ===", 10, 0
    p_misc              db "Miscellaneous Fee                 : ", 0
    p_id_val            db "ID Validation Fee                 : ", 0
    p_lab_ccs0021       db "ITE Computer Lab Fee (CCS0021)    : ", 0
    p_lab_cs0003        db "ITE Computer Lab Fee (CS0003)     : ", 0
    p_lab_cs0070        db "ITE Computer Lab Fee (CS0070)     : ", 0
    p_lab_ged0081       db "Science Lab Fee (GED0081)         : ", 0

    ; --- Labels for Printing the Top Part ---
    lbl_status          db "Enrollment Status : ", 0
    lbl_student_no      db "Student # : ", 0
    lbl_college         db "College : ", 0
    lbl_program         db "   Program : ", 0
```

```
    lbl_name        db "Name : ", 0
    lbl_year        db "Year Level : ", 0
    lbl_space3      db "   ", 0
    lbl_phone       db "Phone : ", 0
    lbl_zip         db "   Zip Code : ", 0
    lbl_address     db "Address : ", 0
    lbl_class       db "Classification : ", 0
    newline         db 10, 0

    ; --- Fixed Schedule Table (Middle Part - Just prints directly) ---
    schedule_table db 10
        db
"+-------------------------------------------------------------------------------------------------------------
--------------------------+", 10
        db "| Courses   Title                          Section   Units   Days    Time
Room          |", 10
        db
"+-------------------------------------------------------------------------------------------------------------
--------------------------+", 10
        db "| CCS0021   INFORMATION MANAGEMENT (LEC)          TN24       2     T
16:00:00-18:40:00                   ONLINE        |", 10
        db "| CCS0021L  INFORMATION MANAGEMENT (LAB)          TN24       1     F
16:00:00-18:50:00                   ONLINE        |", 10
        db "| CS0001    DISCRETE STRUCTURES 1                 TN24       3     S / W
15:00:00-16:50:00 / 15:00:00-16:50:00   F710 / F710    |", 10
        db "| CS0003    COMPUTER SYSTEMS & ARCHITECTURE       TN24       2
F       10:00:00-12:40:00                   ONLINE        |", 10
        db "| CS0003L   COMPUTER SYSTEMS & ARCHITECTURE (LAB)   TN24
1     TH      10:00:00-12:50:00                   E610          |", 10
        db "| CS0070    OBJECT ORIENTED PROGRAMMING           TN24       2     F
07:00:00-09:40:00                   ONLINE        |", 10
        db "| CS0070L   OBJECT ORIENTED PROGRAMMING LAB       TN24       1
TH      07:00:00-09:50:00                   E609          |", 10
        db "| GED0021   SPECIALIZED ENGLISH PROGRAM 2         TN24       3     S /
W   11:00:00-12:50:00 / 11:00:00-12:50:00   F607 / F607    |", 10
        db "| GED0085   MATHEMATICAL ANALYSIS                 MS03       3     S / W
09:00:00-10:50:00 / 09:00:00-10:50:00   F712 / F712    |", 10
        db "| GED0081   COLLEGE PHYSICS 1 LECTURE             TN22       2     F
13:00:00-15:40:00                   ONLINE        |", 10
        db "| GED0081L  COLLEGE PHYSICS 1 LABORATORY          TN22       1
TH      13:00:00-15:50:00                   F1003         |", 10
        db "|
|", 10
        db "| TOTAL UNITS 21
|", 10
        db
"+-------------------------------------------------------------------------------------------------------------
--------------------------+", 10, 0
```

```
    ; --- Labels for Printing the Fees & Installments (Bottom Part) ---
    box_border          db "+-------------------------------------------------------------------------+",
10, 0
    box_empty           db "|", 10, 0
    out_tuition         db "| Tuition Fee (21)                                    ", 0
    out_misc            db "| Miscellaneous Fee                                    ", 0
    out_id_val          db "| ID Validation Fee                                    ", 0
    out_lab_ccs0021     db "| ITE Computer Laboratory Fee (CCS0021)                 ", 0
    out_lab_cs0003      db "| ITE Computer Laboratory Fee (CS0003)                  ", 0
    out_lab_cs0070      db "| ITE Computer Laboratory Fee (CS0070)                  ", 0
    out_lab_ged0081     db "| Science Lab Fee (GED0081)                            ", 0
    out_total           db "| TOTAL ASSESSMENT                                     ", 0
    out_down            db "| Down Payment (Upon Enrollment) 40%                    ", 0
    out_midterm         db "| Midterm (Oct 05 - Oct 07 2026) 30%                    ", 0
    out_final           db "| Final (Nov 26 - Dec 03 2026) 30%                      ", 0

    ; Fixed Tuition Fee: 44,268.00
    ; To make math easy without decimals, we multiply by 100 and store it in centavos
(4426800)
    tuition_fee         dd 4426800

; The .bss section reserves empty spaces (variables) where we will save
; the user's typed answers and the calculated math results.
section .bss
    ; --- Empty spaces for Student Info answers ---
    in_status           resb 64
    in_student_no       resb 64
    in_name             resb 128
    in_phone            resb 64
    in_zip              resb 32
    in_class            resb 64
    in_college          resb 64
    in_program          resb 64
    in_year             resb 32
    in_term_sy          resb 64
    in_address          resb 128

    ; --- Temporary spaces for reading and converting numbers ---
    fee_input_buf       resb 64
    num_str_buf         resb 32

    ; --- Variables to store money amounts (in centavos) ---
    misc_fee            resd 1
    id_val_fee          resd 1
    lab_ccs0021         resd 1
    lab_cs0003          resd 1
    lab_cs0070          resd 1
    lab_ged0081         resd 1
    total_assessment    resd 1
```

```
    down_payment        resd 1
    midterm_pay         resd 1
    final_pay           resd 1

;
================================================================================
; MACROS (Shortcuts that paste reusable blocks of code)
;
================================================================================

; Macro 1: Prints a text message on the screen
%macro PRINT_ZSTR 1
    mov esi, %1             ; Put the text address into ESI
    call print_zstr         ; Run the printing procedure
%endmacro

; Macro 2: Asks a question and saves what the user types on the keyboard
; %1 = question text, %2 = where to save the answer, %3 = max characters allowed
%macro PROMPT_TEXT 3
    PRINT_ZSTR %1           ; Print the question
    mov eax, 3              ; 3 tells Linux we want to read from the keyboard
    mov ebx, 0              ; 0 means standard input (keyboard)
    mov ecx, %2             ; Where to store the typed letters
    mov edx, %3             ; Maximum number of letters to read
    int 0x80                ; Tell Linux to read the keyboard now
    mov esi, %2
    call strip_newline      ; Remove the "Enter" key at the end of the typed text
%endmacro

; Macro 3: Asks for a fee, converts the typed text into a number, and saves it
; %1 = question text, %2 = number variable to save it in
%macro PROMPT_FEE 2
    PROMPT_TEXT %1, fee_input_buf, 64
    mov esi, fee_input_buf
    call parse_currency     ; Convert typed text (like "80.00") into a number (8000)
    mov [%2], eax           ; Save the number into the variable
%endmacro

; Macro 4: Prints one row of the fee table (Label + Money Amount + New Line)
%macro PRINT_FEE_ROW 2
    PRINT_ZSTR %1           ; Print the fee name
    mov eax, [%2]           ; Load the money amount
    call print_currency     ; Print the number with commas and a decimal point
    PRINT_ZSTR newline      ; Move to the next line
%endmacro

;
```

```
;=============================================================================
;=============
; MAIN PROGRAM
;
;=============================================================================
;=============
section .text
   global _start

_start:
   ; Step 1: Ask the user for student info and fees
   call get_user_inputs

   ; Step 2: Add up the fees and calculate the 40%, 30%, 30% payments
   call calculate_assessment

   ; Step 3: Print the top part (Student Info)
   call print_top_section

   ; Step 4: Print the middle part (Subjects, Schedule, and Units)
   call print_schedule_section

   ; Step 5: Print the bottom part (Tuition, Fees, Total, and Installments)
   call print_assessment_section

   ; End the program cleanly
   mov eax, 1            ; 1 tells Linux to exit the program
   xor ebx, ebx          ; 0 means no errors happened
   int 0x80

;
;=============================================================================
;=============
; PROCEDURES (Sub-programs that do specific jobs using call and ret)
;
;=============================================================================
;=============


; ----------------------------------------------------------------------------
; Procedure: get_user_inputs
; Asks the user to type in all the student details and fee amounts
; ----------------------------------------------------------------------------
get_user_inputs:
   PRINT_ZSTR hdr_input_top
   PROMPT_TEXT p_status,     in_status,     64
   PROMPT_TEXT p_student_no, in_student_no, 64
   PROMPT_TEXT p_name,       in_name,       128
   PROMPT_TEXT p_phone,      in_phone,      64
   PROMPT_TEXT p_zip,        in_zip,        32
```

```
    PROMPT_TEXT p_class,      in_class,      64
    PROMPT_TEXT p_college,    in_college,    64
    PROMPT_TEXT p_program,    in_program,    64
    PROMPT_TEXT p_year,       in_year,       32
    PROMPT_TEXT p_term_sy,    in_term_sy,    64
    PROMPT_TEXT p_address,    in_address,    128

    PRINT_ZSTR hdr_input_fees
    PROMPT_FEE p_misc,        misc_fee
    PROMPT_FEE p_id_val,      id_val_fee
    PROMPT_FEE p_lab_ccs0021, lab_ccs0021
    PROMPT_FEE p_lab_cs0003,  lab_cs0003
    PROMPT_FEE p_lab_cs0070,  lab_cs0070
    PROMPT_FEE p_lab_ged0081, lab_ged0081
    ret                   ; Go back to main program

; ---------------------------------------------------------------------------
; Procedure: calculate_assessment
; Adds all fees together and computes the Down Payment, Midterm, and Final
; ---------------------------------------------------------------------------
calculate_assessment:
    ; Start with the fixed tuition fee, then add each user-entered fee
    mov eax, [tuition_fee]
    add eax, [misc_fee]
    add eax, [id_val_fee]
    add eax, [lab_ccs0021]
    add eax, [lab_cs0003]
    add eax, [lab_cs0070]
    add eax, [lab_ged0081]
    mov [total_assessment], eax     ; Save the grand total

    ; Calculate Down Payment (40%): (Total * 40) / 100
    mov eax, [total_assessment]
    mov ebx, 40
    mul ebx                      ; Multiply EAX by 40
    mov ebx, 100
    div ebx                      ; Divide by 100
    mov [down_payment], eax         ; Save Down Payment

    ; Calculate Midterm Payment (30%): (Total * 30) / 100
    mov eax, [total_assessment]
    mov ebx, 30
    mul ebx                      ; Multiply EAX by 30
    mov ebx, 100
    div ebx                      ; Divide by 100
    mov [midterm_pay], eax          ; Save Midterm Payment

    ; Calculate Final Payment (30%): (Total * 30) / 100
    mov eax, [total_assessment]
```

```
    mov ebx, 30
    mul ebx                     ; Multiply EAX by 30
    mov ebx, 100
    div ebx                     ; Divide by 100
    mov [final_pay], eax        ; Save Final Payment
    ret

; ---------------------------------------------------------------------------
; Procedure: print_top_section
; Prints the student's information in two neat columns (Left and Right)
; ---------------------------------------------------------------------------
print_top_section:
    PRINT_ZSTR newline

    ; Line 1: Enrollment Status
    PRINT_ZSTR lbl_status
    PRINT_ZSTR in_status
    PRINT_ZSTR newline

    ; Line 2: Student # on the left, College & Program on the right
    PRINT_ZSTR lbl_student_no
    PRINT_ZSTR in_student_no
    mov esi, in_student_no
    call str_len                ; Count how many letters are in Student #
    add eax, 12                 ; Add 12 for the label "Student # : "
    call pad_to_col_56          ; Print spaces to line up the right column
    PRINT_ZSTR lbl_college
    PRINT_ZSTR in_college
    PRINT_ZSTR lbl_program
    PRINT_ZSTR in_program
    PRINT_ZSTR newline

    ; Line 3: Name on the left, Year Level & Term on the right
    PRINT_ZSTR lbl_name
    PRINT_ZSTR in_name
    mov esi, in_name
    call str_len                ; Count how many letters are in Name
    add eax, 7                  ; Add 7 for the label "Name : "
    call pad_to_col_56          ; Print spaces to line up the right column
    PRINT_ZSTR lbl_year
    PRINT_ZSTR in_year
    PRINT_ZSTR lbl_space3
    PRINT_ZSTR in_term_sy
    PRINT_ZSTR newline

    ; Line 4: Phone & Zip Code on the left, Address on the right
    PRINT_ZSTR lbl_phone
    PRINT_ZSTR in_phone
    PRINT_ZSTR lbl_zip
```

```
    PRINT_ZSTR in_zip
    mov esi, in_phone
    call str_len                ; Get length of Phone
    mov ebx, eax
    mov esi, in_zip
    call str_len                ; Get length of Zip Code
    add eax, ebx                  ; Add Phone + Zip Code lengths together
    add eax, 22                   ; Add 22 for the "Phone : " and "Zip Code : " labels
    call pad_to_col_56            ; Print spaces to line up the right column
    PRINT_ZSTR lbl_address
    PRINT_ZSTR in_address
    PRINT_ZSTR newline

    ; Line 5: Classification
    PRINT_ZSTR lbl_class
    PRINT_ZSTR in_class
    PRINT_ZSTR newline
    ret

; ---------------------------------------------------------------------------
; Procedure: print_schedule_section
; Prints the middle table with subjects, schedules, and units
; ---------------------------------------------------------------------------
print_schedule_section:
    PRINT_ZSTR schedule_table
    ret

; ---------------------------------------------------------------------------
; Procedure: print_assessment_section
; Prints the bottom table with all fees, total assessment, and installments
; ---------------------------------------------------------------------------
print_assessment_section:
    PRINT_ZSTR newline
    PRINT_ZSTR box_border
    PRINT_FEE_ROW out_tuition,      tuition_fee
    PRINT_FEE_ROW out_misc,         misc_fee
    PRINT_FEE_ROW out_id_val,       id_val_fee
    PRINT_FEE_ROW out_lab_ccs0021,  lab_ccs0021
    PRINT_FEE_ROW out_lab_cs0003,   lab_cs0003
    PRINT_FEE_ROW out_lab_cs0070,   lab_cs0070
    PRINT_FEE_ROW out_lab_ged0081,  lab_ged0081
    PRINT_FEE_ROW out_total,        total_assessment
    PRINT_ZSTR box_empty
    PRINT_FEE_ROW out_down,         down_payment
    PRINT_FEE_ROW out_midterm,      midterm_pay
    PRINT_FEE_ROW out_final,        final_pay
    PRINT_ZSTR box_border
    ret
```

```
; ------------------------------------------------------------------------
; Procedure: print_zstr
; Prints any text stored at address ESI until it hits 0
; ------------------------------------------------------------------------
print_zstr:
   pushad               ; Save all current register values
   call str_len         ; Find out how long the text is (length stored in EAX)
   test eax, eax        ; Check if the length is 0
   jz .done             ; If length is 0, skip printing
   mov edx, eax         ; EDX = number of characters to print
   mov ecx, esi         ; ECX = address of the text
   mov ebx, 1           ; 1 means print to the screen (stdout)
   mov eax, 4           ; 4 tells Linux we want to write/print
   int 0x80             ; Tell Linux to print now
.done:
   popad                ; Restore all saved register values
   ret

; ------------------------------------------------------------------------
; Procedure: str_len
; Counts how many characters are in a piece of text
; ------------------------------------------------------------------------
str_len:
   push esi             ; Save ESI so we don't lose the starting position
   xor eax, eax         ; Start our counter (EAX) at 0
.loop:
   cmp byte [esi], 0    ; Did we reach the 0 at the end of the text?
   je .done             ; If yes, stop counting
   inc esi              ; Move to the next letter
   inc eax              ; Add 1 to our counter
   jmp .loop            ; Repeat for the next letter
.done:
   pop esi              ; Restore starting position
   ret

; ------------------------------------------------------------------------
; Procedure: strip_newline
; Removes the "Enter" key press from the end of what the user typed
; ------------------------------------------------------------------------
strip_newline:
   push esi
.loop:
   mov al, [esi]        ; Look at the current letter
   test al, al          ; Is it 0 (end of text)?
   jz .done             ; If yes, we are done
   cmp al, 10           ; Is it the Enter key (New Line)?
   je .replace          ; If yes, erase it
   cmp al, 13           ; Is it a Carriage Return?
   je .replace          ; If yes, erase it
```

```
    inc esi                 ; Otherwise, move to the next letter
    jmp .loop
.replace:
    mov byte [esi], 0       ; Replace the Enter key with 0 to end the text cleanly
.done:
    pop esi
    ret

; ------------------------------------------------------------------------
; Procedure: pad_to_col_56
; Prints blank spaces so the right-side column lines up at position 56
; ------------------------------------------------------------------------
pad_to_col_56:
    pushad
    mov ecx, 56             ; We want the second column to start at space 56
    sub ecx, eax            ; Subtract how many characters we already printed
    cmp ecx, 3              ; Make sure we print at least 3 spaces
    jge .print_spaces
    mov ecx, 3
.print_spaces:
    mov byte [num_str_buf], ' '
    mov byte [num_str_buf + 1], 0
.loop:
    PRINT_ZSTR num_str_buf  ; Print one space
    loop .loop              ; Repeat until ECX reaches 0
    popad
    ret

; ------------------------------------------------------------------------
; Procedure: parse_currency
; Turns typed text like "12,305.00" or "12305" into a number in centavos (1230500)
; ------------------------------------------------------------------------
parse_currency:
    xor eax, eax            ; EAX will hold the whole peso number (starts at 0)
    xor ebx, ebx            ; EBX will hold the centavos part (starts at 0)
    xor ecx, ecx            ; ECX counts how many decimal digits we read
.int_loop:
    movzx edx, byte [esi]   ; Grab one character from the user's input
    test dl, dl             ; Did we reach the end of the text?
    jz .finish
    inc esi                 ; Move pointer to the next character
    cmp dl, ','             ; Ignore commas
    je .int_loop
    cmp dl, ' '             ; Ignore spaces
    je .int_loop
    cmp dl, '.'             ; If we see a decimal point, switch to reading centavos
    je .frac_loop
    cmp dl, '0'             ; Ignore anything that isn't a digit (0-9)
    jb .int_loop
```

```
    cmp dl, '9'
    ja .int_loop
    sub dl, '0'            ; Convert the character ('0'-'9') into an actual number (0-9)
    imul eax, eax, 10      ; Shift existing number left by 1 digit (multiply by 10)
    add eax, edx           ; Add the new digit
    jmp .int_loop

.frac_loop:
    cmp ecx, 2             ; Only read up to 2 digits after the decimal point
    jae .finish
    movzx edx, byte [esi]
    test dl, dl
    jz .finish
    inc esi
    cmp dl, '0'
    jb .frac_loop
    cmp dl, '9'
    ja .frac_loop
    sub dl, '0'            ; Convert character to number
    imul ebx, ebx, 10
    add ebx, edx
    inc ecx                ; Add 1 to our decimal digit counter
    jmp .frac_loop

.finish:
    cmp ecx, 1             ; If the user typed only 1 decimal digit (like ".5")
    jne .combine
    imul ebx, ebx, 10      ; Turn ".5" into 50 centavos
.combine:
    imul eax, eax, 100     ; Convert whole pesos into centavos (multiply by 100)
    add eax, ebx           ; Add the centavos part
    ret

; ------------------------------------------------------------------------
; Procedure: print_currency
; Takes a number in centavos (like 7278100) and prints it nicely as "72,781.00"
; ------------------------------------------------------------------------
print_currency:
    pushad
    ; We build the text backwards from right to left inside num_str_buf
    mov edi, num_str_buf + 31
    mov byte [edi], 0      ; Put 0 at the very end of the buffer

    ; Separate the number into whole Pesos (EAX) and Centavos (EDX) by dividing by 100
    xor edx, edx
    mov ebx, 100
    div ebx

    ; Write the 2 centavo digits and the decimal point '.'
```

```
    push eax              ; Save the whole pesos while we work on centavos
    mov eax, edx          ; Move centavos into EAX
    xor edx, edx
    mov ebx, 10
    div ebx               ; Split centavos into tens (AL) and ones (DL)
    add dl, '0'           ; Convert ones digit to character
    dec edi
    mov [edi], dl         ; Store ones digit
    add al, '0'           ; Convert tens digit to character
    dec edi
    mov [edi], al         ; Store tens digit
    dec edi
    mov byte [edi], '.'   ; Store the decimal point
    pop eax               ; Bring back the whole pesos

    ; Write the whole peso digits and insert a comma every 3 digits
    xor ecx, ecx          ; ECX counts digits so we know when to add a comma
    test eax, eax         ; Check if the peso amount is 0
    jnz .peso_loop
    dec edi
    mov byte [edi], '0'   ; If it is 0, just write '0'
    jmp .out

.peso_loop:
    cmp ecx, 3            ; Have we written 3 digits in a row?
    jne .extract_digit
    dec edi
    mov byte [edi], ','   ; Add a comma!
    xor ecx, ecx          ; Reset the 3-digit counter back to 0
.extract_digit:
    xor edx, edx
    mov ebx, 10
    div ebx               ; Divide by 10: remainder in DL is the rightmost digit
    add dl, '0'           ; Convert digit to text character
    dec edi
    mov [edi], dl         ; Save character into buffer
    inc ecx               ; Add 1 to our 3-digit comma counter
    test eax, eax         ; Are there more digits left?
    jnz .peso_loop        ; If yes, keep looping

.out:
    mov esi, edi          ; Point ESI to the start of our formatted number text
    call print_zstr       ; Print the formatted money amount
    popad
    ret
```